



ClimatePros

Stream 1 Unified Architecture

A Auto Dispatch · B Job Summarisation Copilot · C Parts Finder

Version 1.1 — Draft for client review · Confidential — ClimatePros / techjays

Companion to the Stream 1 Scope Document — the scope defines *what* is built; this document defines *how*.

Document control

Item	Detail
Document	Stream 1 — Unified Architecture
Version	1.1 — Draft for client review
Prepared by	techjays
Classification	Confidential — ClimatePros / techjays
Supersedes	The two per-author drafts of v1.0 — merged, corrected, and condensed into this single document

How to read this document

- Sections are numbered 1–12; cross-references appear as §N — for example §6 Core Logic.
- **Cloud placement rule:** anything AI-specific runs on Azure; everything else runs on AWS with the existing platform; models stay on-prem on ClimatePros hardware. The single cross-cloud hop is the AI request itself (AWS → Azure).
- **SQL Server is reached only through the .NET APIs** — neither the dispatch workers nor the AI server holds a SQL connection: one write path, one auth boundary, one place to audit.
- **Use Case A involves no LLM** — a deterministic, explainable ranking. B and C are AI-assisted; the technician can always override. Open items needing ClimatePros confirmation are collected in the Appendix.

Approval

Approved for ClimatePros: _____ Date: _____

Approved for techjays: _____ Date: _____

Contents

01 Stream 1 Overview	5
A — Auto Dispatch · finding the right technician	5
B — Job Summarisation Copilot · closing out the job	5
C — Parts Finder · finding the part that isn't on the truck	5
02 High-Level Architecture	6
2a · Unified system diagram	6
2b · Detailed architecture per use case	8
03 Technology Stack	10
04 Infrastructure	11
Existing — reused as-is	11
New — introduced by this project	12
Environments	12
Compute sizing	13
Network topology	13
05 Operational Flows	14
A — Auto Dispatch · assignment loop	14
B — Job Summarisation · summarise, review, learn	15
C — Parts Finder · find & allocate	16
06 Core Logic (per use case)	17
A — Auto Dispatch · ranking algorithm	17
B — Job Summarisation · summarise pipeline & retrieval	18
C — Parts Finder · matching & sourcing logic	18
07 Data & Schema	20
A — ElastiCache for Redis structures	20
B — the AI MongoDB collections	20
C — data model	21
08 API Reference	21
A — Auto Dispatch · endpoints	22
B — Job Summarisation · endpoints	22
C — Parts Finder · endpoints	23
09 Observability & Evals	23
Unified observability	24
Evals — the feedback loop (B)	24

10 Cost Projection	25
Cost formulas	25
Unified monthly cost by scenario	25
Database & cross-cloud egress	26
11 Onboarding & Fault Tolerance	26
How every rollout runs	26
A — Auto Dispatch	27
B — Job Summarisation	27
C — Parts Finder	28
12 Non-goals & PII	29
Non-goals	29
PII inventory & data handling	29
Appendix — Open Items	30

01 Stream 1 Overview

ClimatePros runs commercial refrigeration and HVAC field service: ~900–1,000 technicians, 46 branches, 10,000+ sites. Work orders arrive automatically; everything after that still runs on a phone call and a technician's memory. Three use cases, three bottlenecks, one job.

Where AI sits: Use Case A is a deterministic algorithm — no LLM, so every assignment is explainable and repeatable. Use Cases B and C use AI, and the technician can always override it.

A — Auto Dispatch · finding the right technician

Web portal · Dispatcher · deterministic — no LLM

A dispatcher has a job ready and ~40 candidate technicians. The four deciding factors — how far, how familiar, how soon free, how urgent — sit behind separate clicks, and confirming one means dialing candidates one at a time, ~2–3 minutes each. Auto Dispatch ranks on those factors and cascades an accept/decline offer down the shortlist.

B — Job Summarisation Copilot · closing out the job

Mobile · Technician at end of job · AI — technician overrides

Every job ends with a mandatory status change: two free-text notes (customer and office) plus compliance forms — done properly, 10–15 minutes in the field. Thin notes make billing and routing harder and leave the next visit without history.

The Copilot drafts both notes from the technician's rough input, suggests likely form values from similar past jobs, and flags what is missing; the technician reviews, fixes, and submits. It never invents details and never blocks Save. Target: 10–15 minutes down to about 3–4.

C — Parts Finder · finding the part that isn't on the truck

Mobile · Technician on-site · AI — technician overrides

On site, the technician needs a part. Three sources the app already answers — truck, nearby technicians, warehouse; the fourth, a third-party supplier, is a phone call, and the same part carries a different number at every vendor. Miss it and it's a return dispatch.

Parts Finder suggests the likely part with AI and shows, in one search, where it is across all four sources. No price ranking, no ordering. One working day, three assists: (A) the right technician → (C) the missing part → (B) the closeout note (§5).

02 High-Level Architecture

2a · Unified system diagram

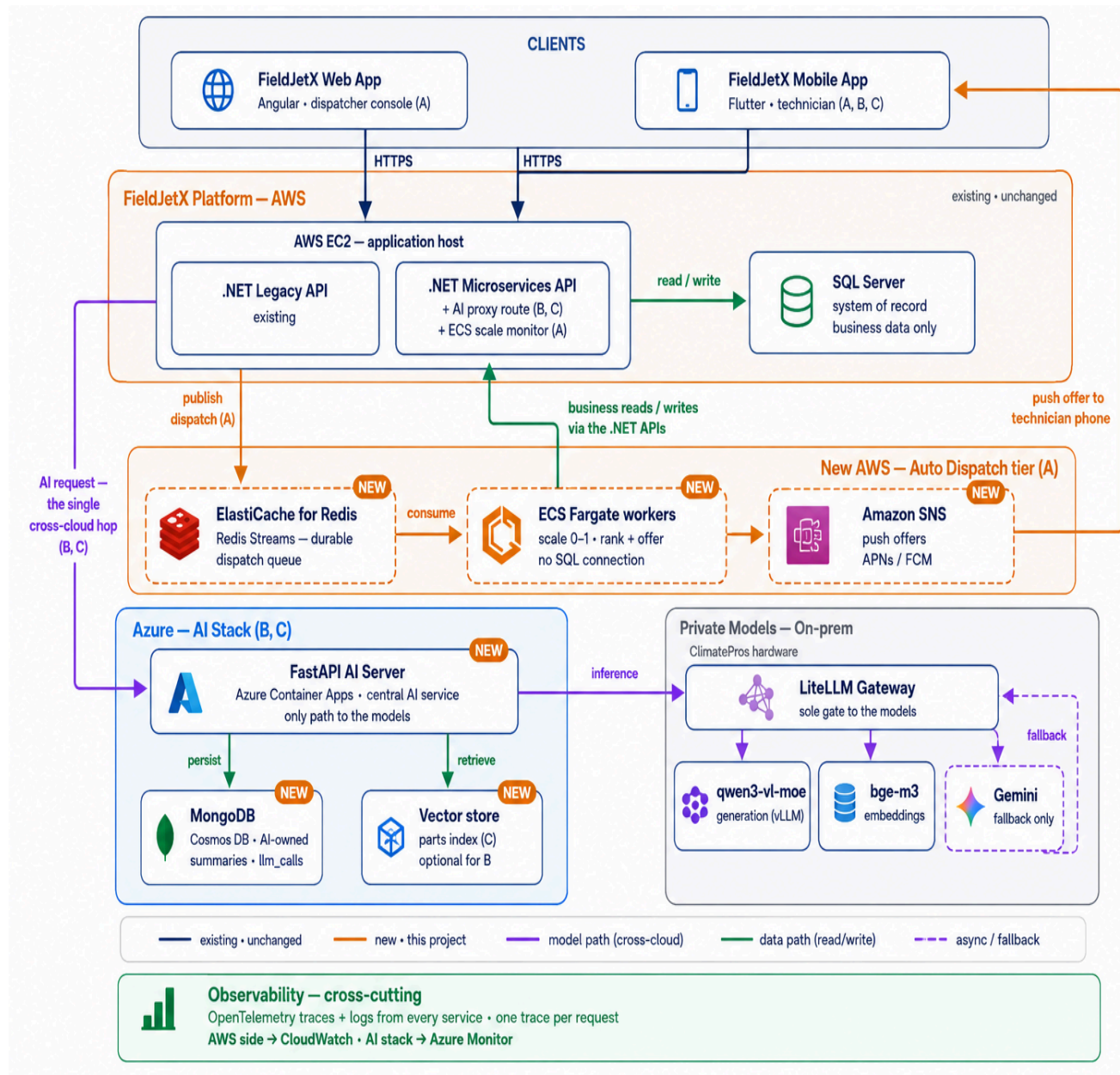


Figure 1 — Unified system diagram · six zones. Workers and the AI server hold no SQL connection; all business data flows through the .NET APIs.

Everything runs on AWS alongside the existing FieldJetX platform — except the AI stack (FastAPI AI server, its own MongoDB, vector store), which runs on Azure, and the models, which stay on-prem. Five relationships carry the whole system:

- **Entry — single front door.** Web and mobile apps speak only to the existing .NET APIs on EC2 over HTTPS. The platform gains exactly two additions: an AI proxy route (B, C) and an ECS scale-monitor route (A).
- **Data — one gate to SQL Server.** SQL Server stays the system of record, read and written only by the .NET APIs on EC2. The ECS dispatch workers exchange all business data — candidates, ranking inputs, assignments — through those APIs, and the AI server owns a separate MongoDB, receiving business context in the request payload. Neither ever connects to SQL Server.
- **Dispatch (A) — entirely AWS, no model.** The .NET API publishes the job to ElastiCache for Redis; the scale monitor spins ECS Fargate 0→1; workers rank technicians and push accept/decline offers through Amazon SNS. The reply returns via the .NET API.
- **AI (B & C) — one cross-cloud hop.** The .NET AI proxy forwards the request, job context in the payload, to the FastAPI AI server on Azure — the single central AI service and the only path to the models.
- **Models — private, on-prem.** All inference goes through the LiteLLM gateway to the self-hosted qwen3-vl-moe (vLLM) and bge-m3 embedding, with Gemini as external fallback only. Every service emits OpenTelemetry traces and logs — CloudWatch on the AWS side, Azure Monitor for the AI stack.

2b · Detailed architecture per use case

A adds: durable dispatch queue + ECS Fargate worker pool + push-offer cascade. The workers consume the Redis stream, coordinate through per-technician locks, and perform every business read and write — including the safe assignment step — through the .NET APIs, which remain the only holder of the SQL connection.

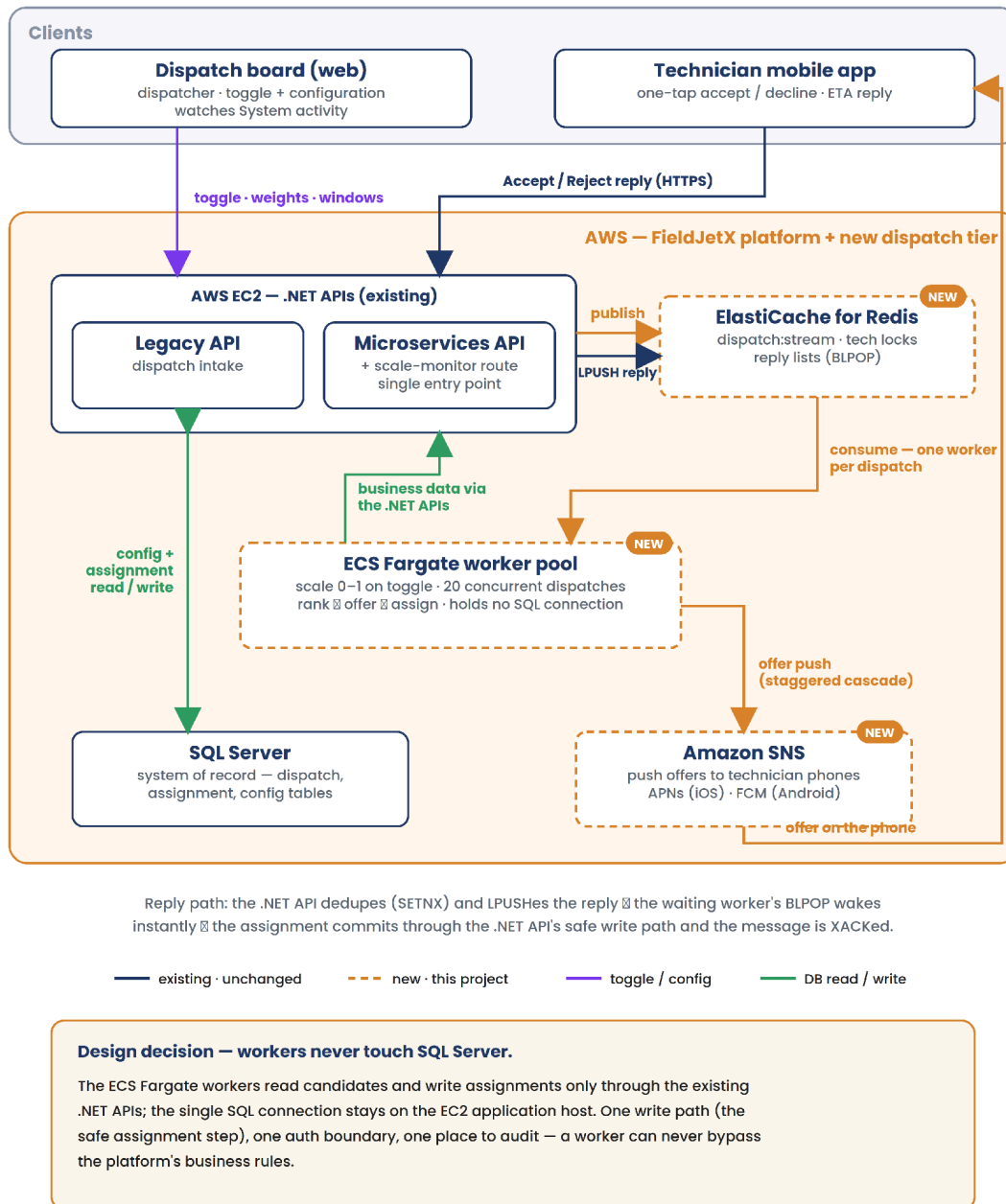


Figure 2 — Use Case A detailed architecture. Entirely within AWS; the worker pool holds no SQL connection.

B & C add: the central FastAPI AI server plus a vector index. B runs the summarise pipeline; C the four-source part search with fuzzy matching — same wiring, different pipeline (§6). The AI datastore is MongoDB on Azure; the vector history index for B is optional, added only if evals demand it.

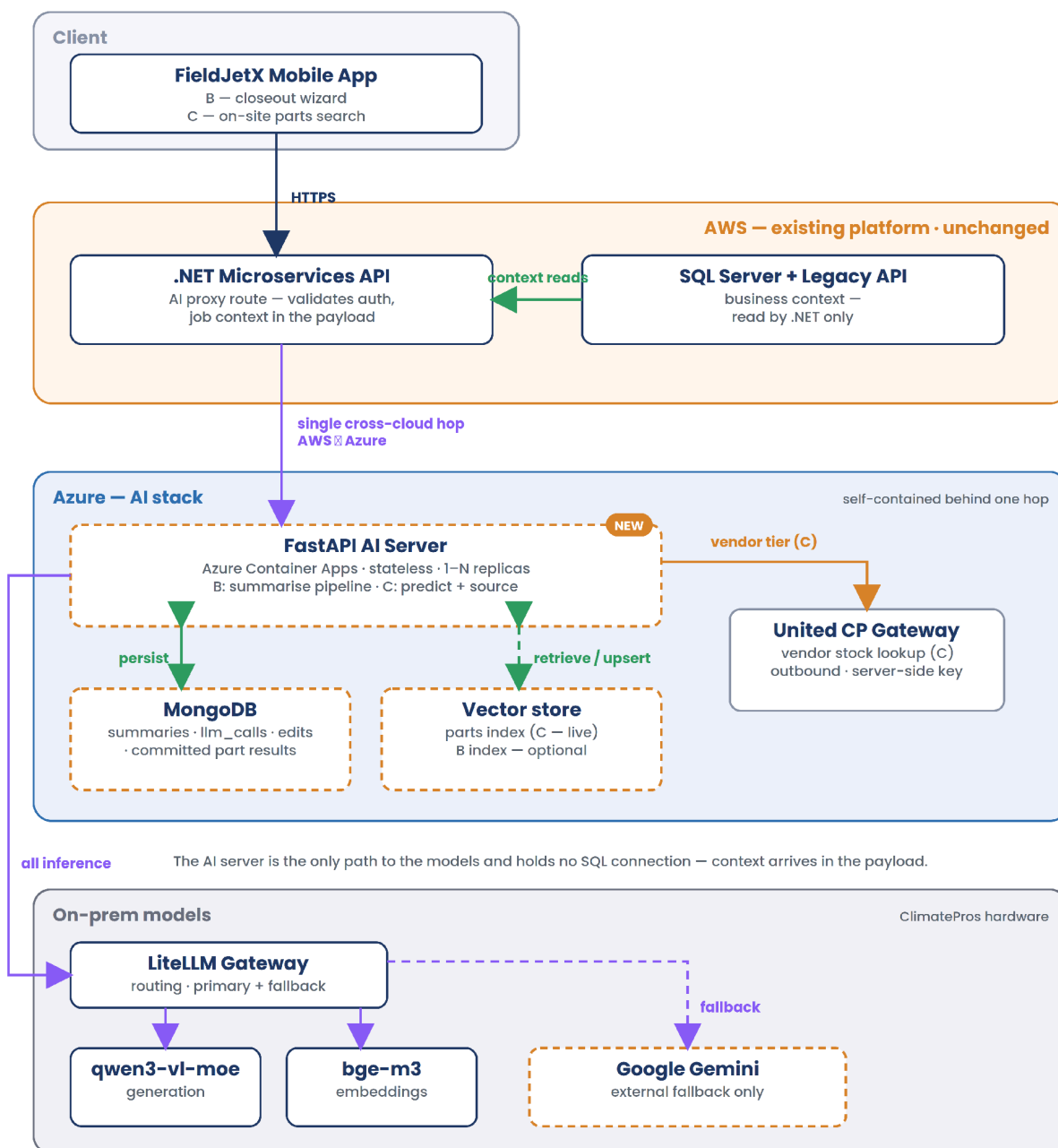


Figure 3 — Use Cases B & C share one AI service; only the pipeline inside it differs.

03 Technology Stack

The existing platform pieces — client apps, the .NET API host, SQL Server, observability — are shared and reused as-is. A's new tier stays on AWS beside the platform; B and C's new tier is the one part of Stream 1 on Azure, reached by a single cross-cloud hop.

Layer	A • Auto Dispatch	B • Job Summarisation	C • Parts Finder
Client	Web dispatch console + technician offers (mobile)	Mobile close-out wizard (Flutter)	Mobile on-site search (Flutter)
App / API host	.NET Legacy + Microservices APIs on AWS EC2 (existing) — one shared host; gains the AI-proxy route (B, C) and the ECS scale-monitor route (A).	← shared	← shared
Central AI service	N/A — no LLM	FastAPI on Azure Container Apps (new) — single entry point for B and C; stateless; only path to the models.	← shared
Workers / broker	ECS Fargate pool (0–1 on toggle) + ElastiCache for Redis Streams (new)	N/A	N/A
Vector store	N/A	Optional similar-incident index — only if evals demand it	Dispatch-parts history index (live)
Push	Amazon SNS — APNs / FCM (new)	N/A	N/A
Database	SQL Server on AWS — system of record, read/written only by the .NET APIs on EC2; workers and the AI server hold no SQL connection. B and C	← shared	← shared

Layer	A · Auto Dispatch	B · Job Summarisation	C · Parts Finder
	persist AI output to the AI server's own MongoDB on Azure.		
Models	N/A	LiteLLM gateway · self-hosted qwen3-vl-moe (vLLM) · bge-m3 embeddings · Gemini fallback — on-prem, client hardware; any model change must pass the eval set first.	← shared
Observability	OpenTelemetry + CloudWatch (AWS) + Azure Monitor (AI stack) — every service emits traces and logs (§9).	← shared	← shared

Dispatch worker stack — A

Python 3.11+ on an asyncio event loop — fully async, no blocking calls. **Push:** aioboto3 against the SNS Publish API, IAM-role credentials. **Messaging:** redis.asyncio — tasks in via Redis Streams (XREADGROUP), replies in via per-technician BLPOP. **External calls:** httpx with pooled async connections — all business data via the .NET APIs.

Central AI service stack — B · C

FastAPI + Uvicorn with pydantic schemas. **Model access:** httpx to the LiteLLM gateway — generation on qwen3-vl-moe, embeddings on bge-m3, Gemini fallback handled by the gateway. **Retrieval:** async client to the Azure vector store; cosine similarity over bge-m3 embeddings.

04 Infrastructure

Existing — reused as-is

- FieldJetX Mobile App (Flutter, B/C) and Web App (dispatcher console, A).
- AWS EC2 host running the .NET Legacy and Microservices APIs — gains the new routes for A/B/C.
- SQL Server on AWS — system of record, business data only, reached only from the EC2 APIs.

New — introduced by this project

- **ElastiCache for Redis** (AWS) — Redis Streams as the durable dispatch queue (A).
- **ECS Fargate workers** (AWS) — scale 0–1 on toggle; consume the queue; business data via the .NET APIs.
- **Amazon SNS** (AWS) — outbound push to technician phones (APNs / FCM).
- **Azure Container Apps — FastAPI AI server** — central AI service for B & C; the only component that reaches the models; also serves the observability/evals dashboard via the .NET proxy.
- **MongoDB (Azure Cosmos DB, vCore)** — AI-owned datastore: summaries · part results · llm_calls · evals. No SQL Server coupling.
- **Vector store (Azure)** — dispatch-parts index (C); optional similar-incident index (B), only if evals demand it.
- **LiteLLM gateway + self-hosted models** (on-prem) — qwen3-vl-moe (vLLM) + bge-m3; Google Gemini as managed fallback only.

Environments

Environment	Purpose	What differs from Prod
Dev	Integration and feature work	Lowest replica caps; Gemini-only models (no on-prem link); points at a SQL snapshot
Staging	Pre-prod validation and evals; mirrors Prod topology	Smaller replica ranges; restricted SQL replica; full model stack for eval fidelity

Environment	Purpose	What differs from Prod
Prod	Live — 900–1,000 technicians, 46 branches	—

Each environment is fully isolated — separate queues, databases, indexes and keys; lower environments never point at live prod data (§12).

Compute sizing

Component	Instance size	Scaling	Notes
Dispatch workers (ECS Fargate) — AWS	0.5 vCPU / 1 GB	0–1 on toggle	60+ concurrent dispatches per instance
AI server (FastAPI) — Azure	0.5 vCPU / 1 GB	1–N on request concurrency	Min 1 replica avoids cold-start on the model path
ElastiCache for Redis — AWS	Managed tier	Fixed; sized to queue depth	Persistence on; no-eviction policy
Vector store — Azure	Managed / index-sized	Grows with parts corpus	3–4K embeddings — small, flat
MongoDB (Cosmos vCore) — Azure	M10 (2 vCore / 8 GB)	Fixed tier; manual step-up	AI stack's own store

Network topology

All existing and new non-AI infrastructure sits in one AWS VPC: EC2 and ECS Fargate in an apps subnet, ElastiCache and SQL Server on private subnets with no public IP. The only public entry is the .NET API behind a load balancer / WAF. A separate Azure VNet holds only the AI stack, with MongoDB and the vector store on private endpoints; technician phones never reach either side directly.

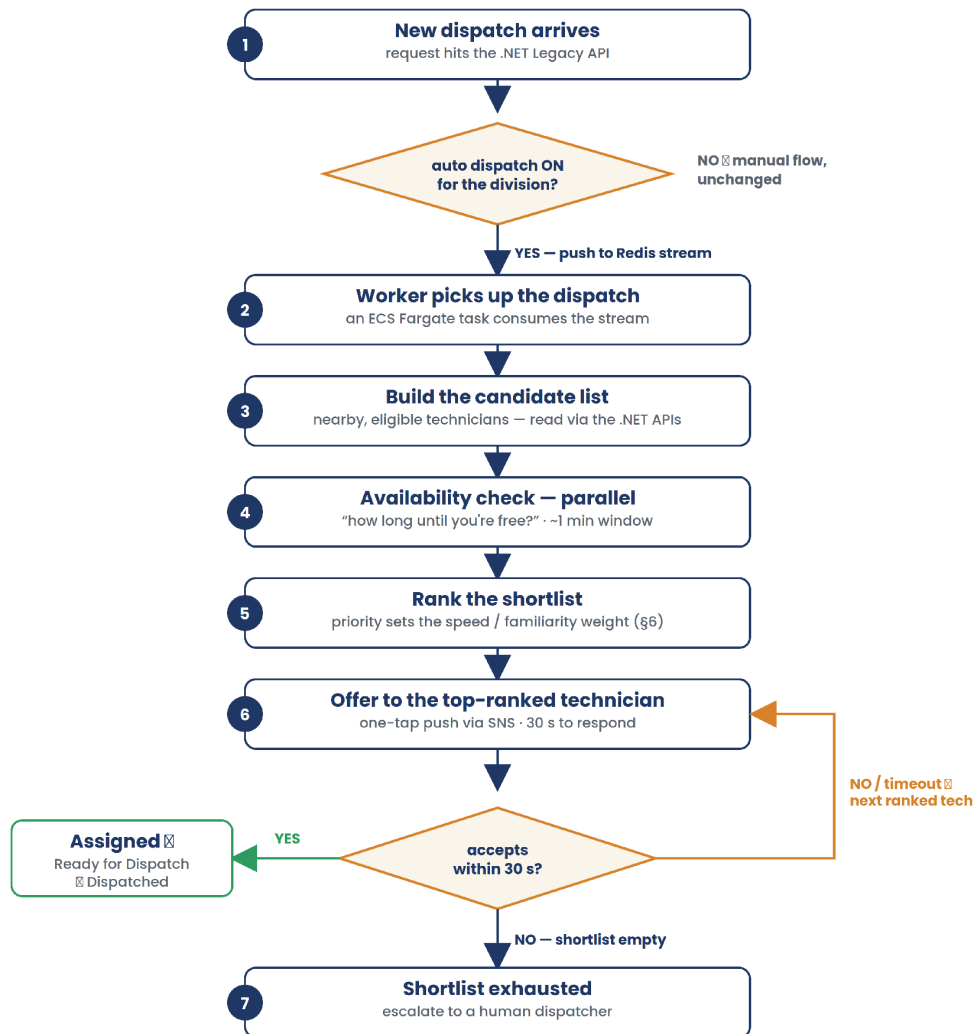
- **The single seam is the AI request** — the .NET proxy on AWS calling the FastAPI server on Azure. Target state is a site-to-site VPN or VPC-to-VNet peering; the interim fallback is a locked-down public HTTPS endpoint (IP allow-list / mTLS).

- **If the link goes down**, B and C fall back to their manual paths; Auto Dispatch (A) is unaffected — it never touches Azure.
- **Latency and egress** are paid once per B/C request, not per data row — budgeted into the P95 targets (§9) and the egress line (§10).

05 Operational Flows

A — Auto Dispatch · assignment loop

A new job enters the queue only for divisions with Auto Dispatch on, then works down a ranked shortlist until a technician accepts or the shortlist runs out. The reply-wait timeout stays below the technician-lock TTL, so a lock is never lost mid-flight — on timeout the offer simply advances to the next ranked technician.

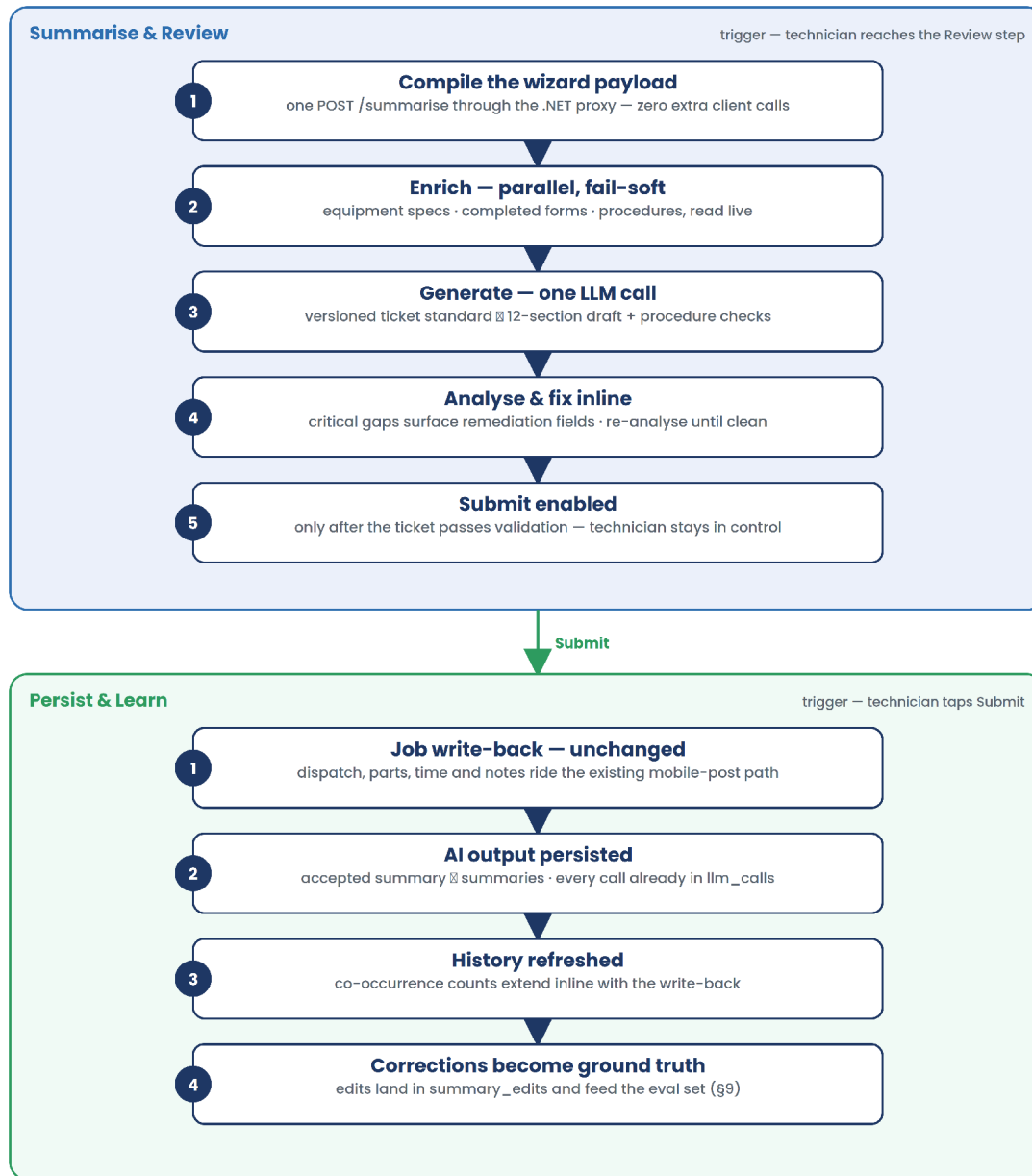


Timeout rule: the reply-wait timeout stays below the technician-lock TTL, so a lock is never lost mid-flight.

Figure 4 — One dispatch, trigger to outcome: ranked cascade, 30-second offer window, human escalation.

B — Job Summarisation · summarise, review, learn

One POST through the .NET proxy carries the whole wizard payload; a single LLM call drafts the 12-section ticket plus procedure checks; gaps re-analyse until clean. On Submit, the write-back rides the existing path unchanged, AI output persists to the AI MongoDB, and edits become eval ground truth (§9). A site briefing at assignment time is out of scope now.



The flywheel: every clean ticket sharpens the history the next technician's ticket draws from.
The AI drafts; the technician reviews, edits and submits — nothing is invented, and Save is never blocked.

Figure 5 — The two loops the copilot runs: Summarise & Review at the Review step, Persist & Learn on Submit.

C — Parts Finder · find & allocate

For every requested part, the Source API returns every place it can be collected, free own stock first; the needed quantity splits greedily down the sorted list until met. Prediction (which parts the job likely needs) runs before sourcing — the pipeline is §6.

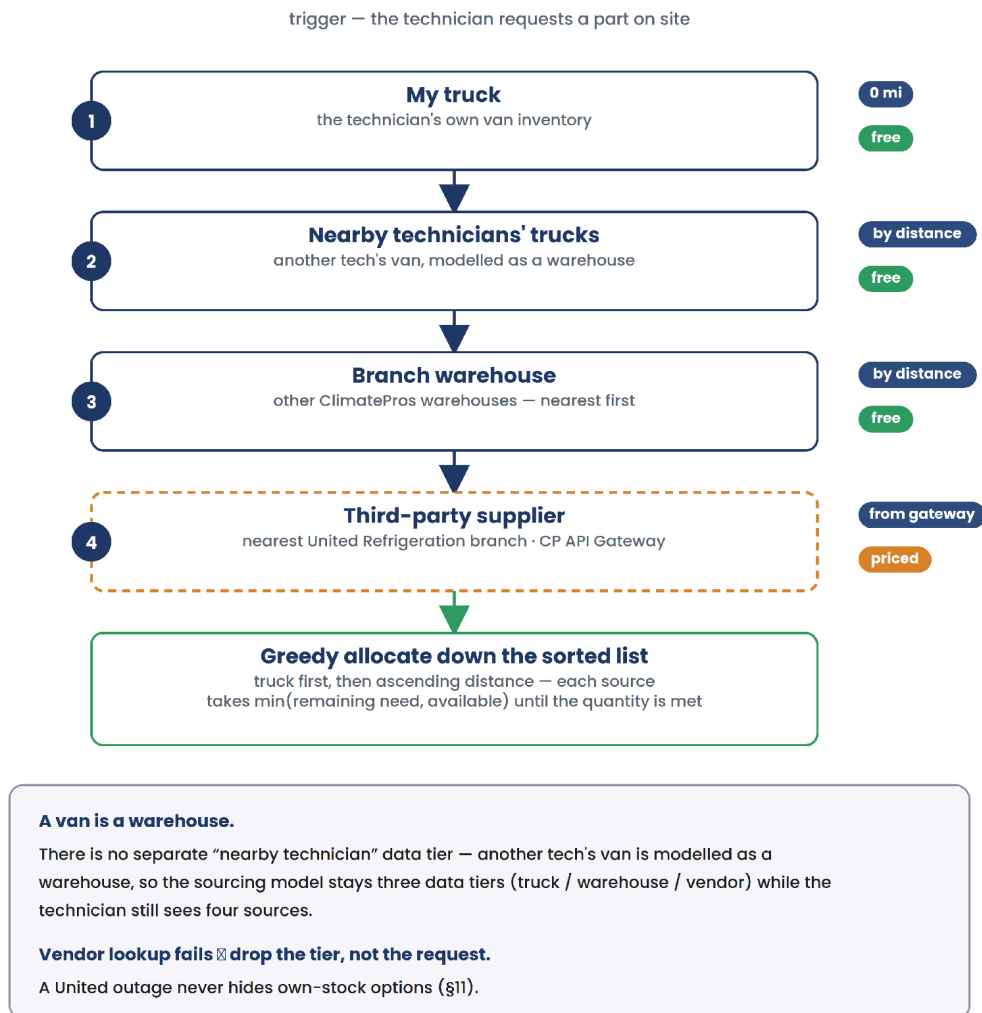


Figure 6 — Four sources, ordered so free stock comes first; the vendor tier is paid and dropped independently on failure.

06 Core Logic (per use case)

A — Auto Dispatch · ranking algorithm

The detail behind “rank the shortlist” in §5 — availability, distance, and visit history collapse into one ordered list. One formula covers every case; a single weight w , chosen from the dispatch type, tunes its behaviour.

Inputs: $free_i$ — minutes until technician i is free; $dist_i$ — miles to the jobsite; $visits_i$ — prior visits to this customer; AVG_SPEED — configurable truck speed; $dispatchType$ — Emergency · High · Medium.

Step A — unify availability + distance into one ETA

```
travel_i = (dist_i / AVG_SPEED) * 60      # miles -> minutes
eta_i    = free_i + travel_i              # minutes until on site - lower is better
```

Step B — weighted score

```
speed_score_i = (eta_max - eta_i) / (eta_max - eta_min)      # lowest eta -> 1.0
fam_score_i   = (visits_i - visits_min) / (visits_max - visits_min)
score_i       = w * speed_score_i + (1 - w) * fam_score_i    # w = speed weight
# sort score_i descending -> rank 1 = offer target
```

Edge cases: a single candidate is ranked 1 by default; if all ETAs tie, familiarity breaks the tie (and vice-versa).

Step C — pick w from the dispatch type

Dispatch type	Strategy	w (speed)	Result
Emergency	Speed-based (locked)	1.0	Pure ETA — familiarity ignored
High Priority	Combined	0.2 (Speed 20 / Fam 80)	Weighted blend; configurable per strategy
Medium Priority	Familiarity-based	0.0	Pure visit count — most familiar wins

One formula, three behaviours — no separate ranking code per type. **Future skill gate:** when technician skill data exists, a job's minimum skill enters as a hard filter on the candidate list, not a score term; the scoring is untouched.

B — Job Summarisation · summarise pipeline & retrieval

One LLM call bracketed by deterministic reads before it and deterministic rules after it — one path, every dispatch. **Inputs:** the wizard answers (reason, parts, time, notes, forms); live context (equipment specs, completed forms, procedures); the versioned ticket_standard.md prompt template; and the SQL co-occurrence history.

- **Compile & forward** — mobile app → .NET proxy (validates auth) → FastAPI, one POST — no extra client round-trips.
- **Enrich, prompt, generate** — parallel fail-soft context reads → one prompt from ticket_standard.md → one LLM call (~6–7 s) → draft 12-section ticket + procedure checks as structured data.
- **Normalise & submit** — draft mapped onto the fixed 12 sections (a missing section becomes an explicit placeholder, never dropped); deterministic rules override sections already known from data; the technician fixes remaining gaps inline; submit writes through the existing path.

Need	Mechanism
Part / form-value suggestions	SQL co-occurrence: (equipment type × problem code) → most-used parts, falling back to equipment type alone, then global top parts — one aggregation, fully explainable
Compliance procedures	Always fetched live from the source system, never cached or embedded — compliance-critical and subject to change

C — Parts Finder · matching & sourcing logic

Prediction is retrieval-augmented: find the most similar past dispatches, then rank the parts those dispatches used, weighting closeness over raw frequency so one near-identical job outranks several loosely-similar ones.

- **Asymmetric embedding** — historical reasons embed at index time (document task type); the live dispatch reason embeds per request (query task type); both on the self-hosted embedding model — dispatch text never leaves client infrastructure.

- **Retrieve** — top K = 10 nearest neighbours from the vector store, with cosine distances and each neighbour's partsUsed.
- **Rank, aggregate, select** — convert distance to similarity, gate on a threshold, accumulate a cubed-similarity score per part, normalise to a confidence band, return the top parts and pre-select the confident ones.

```
sim = 1 - cosine_distance
if sim > 0.55:
    # SIM_THRESHOLD - neighbour contributes
    for part in neighbor.partsUsed:
        score[part] += sim ** 3 # closeness > frequency
        timesUsed[part] += 1
confidence(part) = clamp(round(score[part] / max_score * 95), 20, 95)
return top 6 parts by score # pre-selected if confidence >= 60
```

Constant	Value	Effect
TOP_K	10	Neighbours retrieved per query
SIM_THRESHOLD	0.55	Weaker matches are ignored
Weight	sim ³	Sharply favours very close matches
Confidence band	[20, 95]	Normalised to the top score × 95, then clamped
Output / pre-select	Top 6 · ≥ 60	Returned to the technician; confident parts pre-selected

Indexing & the self-learning loop: idempotent upsert keyed by dispatchNo; rate-limit-aware batching with retry/backoff; when a dispatch completes, its reason and used parts are embedded and upserted — the very next similar job benefits. The response also reports basedOn — how many neighbours cleared the threshold.

Tier	What it is	Distance	Price
truck	The technician's own van inventory	0 mi	free
warehouse	Other ClimatePros warehouses — including other technicians' vans — plus the branch warehouse	nearest first	free
vendor	Nearest United Refrigeration branch (CP API Gateway)	from gateway	priced

A van is a warehouse — three data tiers behind the four sources the technician sees.

Ordering & allocation: truck first (0 mi), then ascending distance; own stock carries no price, only the vendor tier is priced (`InventorySupplierXREF.currentcost`); a large need is split greedily — each source takes `min(remaining need, available)` until the quantity is met. A failed vendor lookup drops that tier, never the request (§11).

07 Data & Schema

A coordinates purely through ephemeral Redis structures — no data model of its own. B and C persist AI output into the AI server's own MongoDB on Azure, with zero coupling to the business tables; C additionally maintains a derived vector index.

A — ElastiCache for Redis structures

Structure	Key	Purpose
STREAM	<code>dispatch:stream</code>	Durable buffer between the .NET API and the workers — consumer group (<code>XREADGROUP</code>); each message goes to exactly one worker
MUTEX	<code>lock:tech:{techId}</code>	Per-technician lock, acquired in fixed order to avoid deadlock — a technician is never double-offered; TTL self-releases on a crashed worker
LIST	<code>resp:{dispatchId}:{techId}</code>	Per-technician reply channel — the API LPUSHes, the worker BLPOPs (instant wake, no polling); EXPIRE 60 s self-cleans

A worker processes up to 20 dispatches concurrently (`redis.asyncio`) — a blocked BLPOP on one dispatch never stalls the others.

B — the AI MongoDB collections

Collection	Purpose
<code>summaries</code>	The accepted 12-section ticket + procedure-check results per dispatch — keyed by the <code>dispatchId</code> carried in the payload, not a database foreign key

Collection	Purpose
llm_calls	One record per model call: provider, model, tokens in/out, latency, outcome — feeds cost tracking and the dashboard (§9)
summary_edits	Every human correction to an AI-drafted section — the eval ground truth (§9)

Indicative collections — final schema and index design is agreed with the client DBA (Appendix).

C — data model

Entity (SQL Server — read only)	Purpose in this system
Dispatch	reason is the prediction input; completion feeds the learning loop
DispatchPartsRequest / Inventory	Parts catalogue and the parts a dispatch requests/uses
InventoryLocationXref	Stock by location — drives own-stock availability in sourcing
InventorySupplierXREF	currentcost — the vendor price applied to the United tier

AI writes land only in the AI server's MongoDB (committed parts lists, acceptance signals, indexing bookkeeping). The vector index holds one record per completed dispatch — id = dispatchNo, the embedded reason text, and partsUsed metadata — **derived and rebuildable**; the relational database remains the source of truth. Because the AI server has no SQL connection, dispatch completion is pushed to it by the .NET side through the AI-proxy route — confirming that trigger is an open item (Appendix).

08 API Reference

The HTTP surface, kept deliberately brief — endpoints, callers, and purpose only. Request/response schemas live in the accompanying API specification. All existing endpoints are unchanged and out of scope here.

A — Auto Dispatch · endpoints

Method	Path	Caller	Purpose
POST	/divisions/{id}/toggle	Dashboard	Auto Dispatch ON/OFF per division — enable pulls pending work into the stream and scales the pool up; disable drains to zero
PUT / GET	/divisions/{id}/config	Dashboard	Read/update per-division ranking weights and windows — applied to the next loop, no restart
POST	/devices/register	Mobile	Register/refresh a push token — upsert on (techId, token); re-posted on OS token rotation
POST	/technicians/{id}/availability	Mobile	“When will you be free” reply — updates general availability read by ranking
POST	/dispatches/{id}/respond	Mobile	Accept/reject one specific offer — deduped, then LPUSHed to the waiting worker

Every endpoint sits behind the platform's existing auth middleware — admin Dashboard auth on the control plane, technician mobile auth on the data plane (identity must match the techId in the path or body). All endpoints return the platform's standard error envelope (stable machine code, human-readable message, correlation requestId) with the usual 400/401/403/404/409/500 semantics.

B — Job Summarisation · endpoints

Method	Route	Purpose	Latency
POST	/api/v1/summarise	Wizard payload → 12-section ticket + procedure checks	~6–7 s
POST	/api/v1/analyse	Ticket → gap findings + inline remediation; re-run until clean	~6–7 s

Method	Route	Purpose	Latency
GET	/api/v1/suggestions	Top-k part suggestions from the co-occurrence tables — deterministic, no LLM	< 200 ms
GET	/admin/dashboard	Edit-rate, latency, cost, parse failures — read from the AI MongoDB	< 500 ms
GET	/health	Liveness probe	< 50 ms

No client calls these routes directly — the .NET proxy is the only caller (internal API key + private VNet); the dashboard is a .NET-proxied route with an admin-role check on the existing Entra token. /summarise and /analyse are read-only and safe to retry.

C — Parts Finder · endpoints

Method	Route	Purpose
POST	/parts/predict	RAG-predicted parts for a dispatch — via the AI proxy route; 503 when prediction is unavailable (client falls back to manual add)
POST	/parts/source	All sources per confirmed part, grouped by tier with quantity and distance — via the AI proxy route
GET	/health	Liveness / deploy probe — not proxied to clients

The vendor tier calls the ClimatePros CP API Gateway for United Refrigeration server-side — the subscription key never leaves the server. Both routes are read-only, side-effect free, and safe to retry; dispatch list/detail and the manual-add inventory tabs stay on the existing .NET APIs, unchanged.

09 Observability & Evals

One common chapter for all three use cases. Observability is operational and unified — one tracing/logging layer watches everything the same way. Evals apply only where a model is in the loop: A is deterministic with nothing to eval; B has a production-grounded eval loop; C validates retrieval quality on a held-out set, with a correction-based loop as future work.

Unified observability

- **Distributed traces** — one trace per request across every hop (.NET → Redis → worker → .NET, or .NET → Azure → LiteLLM → model); the AWS→Azure hop is a span like any other, so its round-trip is visible.
- **Structured logs** — one schema; AWS services report to CloudWatch, the AI stack to Azure Monitor.
- **Correlation** — the requestId from the error envelope joins traces, logs, and the AI collections (llm_calls, summary_edits), so a slow trace, a log line, and an eval finding resolve to one id.

P95 targets — open item: no formal SLOs exist yet. Known inputs: the ~1 min availability window and BLPOP timeout (A), the ~6–7 s LLM-bound calls (B), and the <200 ms deterministic endpoints (B/C). Numeric per-flow targets are set once Azure Monitor has real traffic to baseline against.

Evals — the feedback loop (B)

- llm_calls records provider, model, tokens, latency and outcome for every call — cost and reliability queryable at any time.
- summary_edits records every human correction to an AI section — labelled ground truth from normal usage.
- The eval set is rebuilt from those corrections and re-run before any prompt, ticket-standard, or model change ships.
- **Headline metric:** edit-rate per section, tracked over time and per customer.

Prompts build from a versioned ticket_standard.md and every call logs its model id, so any production ticket traces back to the exact standard version and model. The dashboard is served by the same FastAPI container from the AI MongoDB — no separate web app, proxied through .NET with an admin-role check. One caveat: the structured outcome codes in the history are weak signal (problem code OTHER at 38%, summary code SERVICE at 88% of filled rows) — summary_edits works around this by labelling the model's actual output, but the eval set is only as good as what technicians correct.

10 Cost Projection

One shared infrastructure bill across A/B/C — same compute tiers, same fixed floor. A use case appears only as a load driver (dispatch hours-on, B+C call volume), not as a separate section. Figures are for one environment; Dev/Staging each duplicate the bill.

Pending re-pricing

Azure lines (Container Apps, vector store, Monitor) carry AWS rate-card estimates, not real Azure prices yet; Cosmos DB is a real Azure quote, unverified for region/tier. Re-price before using this for a budget.

Cost formulas

```
# ECS Fargate – dispatch workers (0.5 vCPU / 1 GB)
per_task_hour = 0.5 x $0.04048 + 1 x $0.004445 = $0.024685
monthly_cost = N x H x 30 x per_task_hour      # N = tasks (0-1), H = hours ON/day

# fixed / near-fixed lines
ElastiCache Redis      = $0.032/hr x 730      = ~$23.36/mo
SNS push (75k pushes/mo) =                    ~$0.04/mo
Container Apps (AI)     =                      $20-$40/mo (1-2 replicas)
Vector store            =                      ~$5/mo flat (3-4K embeddings)
Azure Monitor + OTel    =                      $15-$30/mo
Cosmos DB Mongo M10     = $0.129/hr x 730      = ~$94/mo flat
```

Unified monthly cost by scenario

Component	Light (2 h/day)	Typical (8 h/day)	Heavy (24×7)
Dispatch workers — Fargate	~\$1.48	~\$5.92	~\$17.77
AI server — Container Apps	~\$20	~\$30	~\$40
Redis (fixed)	~\$23.36	~\$23.36	~\$23.36
SNS push	~\$0.04	~\$0.04	~\$0.04
Vector store (flat)	~\$5	~\$5	~\$5
Monitor + OTel	~\$15	~\$22	~\$30
MongoDB — Cosmos vCore (flat)	~\$94	~\$94	~\$94

Component	Light (2 h/day)	Typical (8 h/day)	Heavy (24×7)
Variable — per B+C call	< \$1	~\$2–\$5	~\$10–\$25
Total / mo (est.)	~\$159	~\$182–\$185	~\$220–\$235

Fixed floor (Redis + vector store + Mongo + Monitor) ≈ \$137–\$152/mo regardless of traffic; only Fargate, Container Apps, and the per-call variable scale toward zero. Self-hosted models carry no per-token fee — the marginal cost of a B/C call is a fraction of a cent.

Database & cross-cloud egress

SQL Server stays on AWS, unchanged — existing and sunk, no incremental cost and no egress (only AWS-side compute connects to it). The one cross-cloud line is the AWS→Azure AI request/response (B/C): one pair per call, not a per-row transfer. Sizing needs a traffic estimate — get an AWS Data Transfer OUT quote once real volume is known.

11 Onboarding & Fault Tolerance

Two questions per use case: how it reaches live traffic from an empty environment, and how it behaves when a dependency fails. The rollout shape is identical for all three; deployment rides the client's existing pipelines — Stream 1 adds no new deployment tooling.

How every rollout runs

Phase	Purpose	Gate before the next phase
0 · Provision	Stand up what the feature cannot start without — network, hosting, queues, stores, credentials	Every dependency reachable and health-checked
1 · Seed	Load what the logic reads on its first request — config, schema, indexes, history	Nothing is empty at first call — no cold start
2 · Roll out	Ship behind a flag or app-version gate, pilot cohort only	Pilot cohort live; everyone else unchanged

Phase	Purpose	Gate before the next phase
3 · Go live	Monitoring first, then one division/branch, then expand	Pilot clean on its own metrics before widening

A — Auto Dispatch

Onboarding: Redis cluster + consumer group, worker image and scale policy, SNS platform apps (APNs/FCM), IAM roles (Phase 0); per-division weights seeded with the toggle defaulting OFF and an escalation target defined (Phase 1); push-enabled app build with a minimum-version gate — tokens upsert on (techId, token) and re-register on rotation (Phase 2); pilot one division, validate exactly-once assignment and escalation, then expand (Phase 3). A technician without a push registration is skipped to manual — never offered a job they cannot receive — and rejoins on their next token refresh.

New personal data

The push token and its SNS endpointArn are the only new PII this project stores — encrypted at rest, limited to the dispatch backend, overwritten on rotation, cleaned up when dead (§12).

Failure	Behaviour — and why it is safe
Worker dies mid-dispatch	The request stays flagged unfinished; after the visibility timeout another worker claims and completes it — nothing lost
Worker slow, not dead	It still holds the per-technician claim, so a takeover is blocked — the same job never reaches two technicians
Worker truly crashed	Its claim releases automatically after a short hold and a healthy worker takes over
No technician accepts	The 30-second window cascades down the shortlist, then escalates to a human — a job is never silently dropped

Net effect — exactly-once: every request is processed even if a worker dies; none is processed twice even if a worker is slow.

B — Job Summarisation

Onboarding: private subnet + internal load balancer (no public URL), always-on FastAPI at min 2 instances, model-gateway path and secrets in Key Vault (Phase 0); DBA sign-off

on the read-only SQL account, MongoDB collections + indexes, ticket_standard.md v1 frozen, co-occurrence tables seeded from 12 months of history, pilot customers' procedures verified (Phase 1); copilot build + min-version gate – older app versions keep the manual review page and never call the AI route (Phase 2); monitor LLM latency/error/parse-failure and the edit-rate dashboard, pilot one branch, capture the baseline, then expand – any later prompt or model change must pass the eval set first (Phase 3).

Failure	Behaviour – and why it is safe
An enrichment read fails	Pipeline continues without that block (fail-soft) – missing context costs quality, not availability
Private gateway outage / saturation	AI features degrade to the manual experience; calls past the timeout fall back to the plain recap – submission is never blocked
Malformed model output	Parser falls back to regex extraction, then MISSING placeholders – the app always receives exactly 12 sections
A Container Apps instance dies	The load balancer routes to the remaining instance and the replica is replaced – min 2 always-on, no cold start

SQL Server stays the single source of truth – the AI tier writes only to its own MongoDB, so an AI failure can never corrupt or block business data.

C – Parts Finder

Onboarding: Container Apps on the private VNet, vector index created, connectivity to the embedding model, the .NET APIs and the United gateway health-checked (Phase 0); batch-embed historical dispatches (idempotent upsert by dispatchNo) and validate retrieval on a held-out set – the index is never empty at go-live (Phase 1); ship “Add Parts with AI” behind a per-branch feature flag and pilot a small cohort watching acceptance rate and source mix (Phase 2); expand division by division and wire the on-completion re-embed pipeline so the index grows as dispatches close (Phase 3).

Failure	Behaviour – and why it is safe
Models / vector store unavailable	/parts/predict returns 503; the client falls back to manual add – prediction is an accelerator, not a gate

Failure	Behaviour — and why it is safe
United outage or timeout	The vendor tier is dropped from that part's sources; own-stock options still return — the request does not fail
Model throttling	Retry with backoff and batched pacing at index time; request-time timeouts bound prediction latency
SQL Server slow / replica crash	Brief caching serves last-known stock where safe; stateless replicas are replaced behind the load balancer

12 Non-goals & PII

Non-goals

- **A** — not route optimisation (it ranks who answers next, not a multi-stop route); deterministic, no LLM; the manual dispatch flow is untouched with the toggle OFF; no UI change beyond the toggle and its configuration; no schema change beyond dispatch status fields.
- **B** — never fabricates (a missing block degrades to a MISSING placeholder, never an invented one); never blocks Save (every failure mode degrades to the manual recap); completeness over prose — the headline is cross-checking the note against parts, time and photos actually logged.
- **C** — no price ranking, no ordering (the technician decides where to collect); one supplier for the POC (United, via the existing CP gateway); predictions are suggestions, never auto-committed; no purchasing, PO creation, or inventory management — stock stays owned by the system of record.

PII inventory & data handling

Area	Control
Client → service auth	Bearer token from the app session, validated at the .NET API before the AI proxy forwards; CORS restricted to the mobile client
Network isolation	The AI layer runs in a private Azure VNet — never internet-reachable; only the .NET proxy (and private paths to stores and models) can reach it

Area	Control
Model hosting	Embedding and LLM inference run on client-hosted models — dispatch text and parts history are never sent to a third-party AI provider
Supplier gateway	United calls carry a subscription key held server-side only — it never reaches the mobile client
Secrets	All credentials resolved from Azure Key Vault with rotation — none in source or images
Data store access	Least-privilege, read-only SQL account scoped to inventory and dispatch tables; every AI write goes to the AI-owned MongoDB
Transport	TLS on all hops

- Dispatch and customer detail is served to the authenticated technician only and excluded from log bodies; van/warehouse locations are used transiently for distance ordering, not persisted.
- **A:** the push token / endpointArn is the one new piece of personal data — encrypted at rest, access-limited, overwritten on rotation (§11).
- **B:** notes and the correction history carry the same sensitivity as today's manual notes — stored in the AI MongoDB behind the proxy. Unlike C's rebuildable index, they are original records: a cascading-deletion mechanism against SQL-side deletion requests still needs defining (Appendix).
- **C:** the vector store holds only dispatch reason text and used part numbers — derived from SQL Server and re-seedable at any time, so deletion/retention requests are handled at the SQL layer.

The history B and C learn from: 10–12 years of operational data — 200,000+ jobs in the last six months, 16.3M note records — and messy by the client's own admission (~40% of notes $\leq$ 40 characters). The corpus is de-duplicated and PII-stripped before anything reaches a model — a data-preparation step upstream of both §6 pipelines.

Appendix — Open Items

Each needs a ClimatePros owner before this document moves from draft to agreed.

#	Open item	Where	Why it matters
1	Private-runtime / model-gateway readiness — endpoint, credentials, model version, first integration-test date	§2–§4	The privacy-first posture depends on it; capacity figures are client-stated
2	Azure re-pricing for the AI lines (Container Apps, vector store, Monitor)	§10	Current figures are AWS-sourced rates
3	Vector store selection (Azure AI Search / Pinecone) and its C-only scope	§3, §6	C needs a store at go-live; B only if evals justify it
4	Learning-loop trigger for C — dispatch completion must be pushed by the .NET side (the AI server has no SQL connection)	§6, §7	Without it the self-learning index never updates
5	MongoDB collection and index design sign-off with the client DBA	§7	Query performance for retrieval and the eval dashboard
6	Cross-cloud network path — VPN / peering target vs. interim allow-listed endpoint	§4	Security review and the P95 budget both depend on it
7	Push token / endpointArn in the PII inventory and retention policy	§11, §12	The one new piece of personal data this project stores
8	Embedding model name — bge-m3 vs nomic-embed appear in different sources for the same gateway	§3, §4	Onboarding, cost, and eval work all need the real name
9	Gemini fallback vs. the “never a third-party provider” claim — the fallback’s data handling must be covered or the fallback removed	§3, §12	PII posture must be consistent
10	Cascading deletion for B’s summaries/edits against SQL-side deletion requests	§12	They are original records, not rebuildable